Model Predictive Path Integral Control for Car-Like Robots: Implementation, Cost Design and Robustness Evaluation

Mahim Trivedi
Python Robotics Projects, github.com/robofan45/mppi-control
November 2026

***Abstract*—Model Predictive Path Integral (MPPI) control chooses each command by simulating hundreds of randomly perturbed control sequences and averaging them with weights that decay exponentially with their cost. Because it needs neither gradients nor a convex model, it can use blunt, discontinuous costs such as a fixed penalty for every simulated collision. This report presents a compact implementation of MPPI that drives a kinematic bicycle model along a course while avoiding obstacles placed on it, and documents three design decisions that turned out to matter more than any tuning constant. First, sampling the rate of change of the steering angle, with the angle itself kept in the state, bounds the steering motion between control steps by construction. Second, the speed term of the cost must count only the speed along the course: with the raw speed, a car circling beside an obstacle at full speed is cheaper than one driving around it. Third, the control cost of the information-theoretic derivation must not be weighted by the full temperature: with that weight, a median of only 2.6 of the 300 samples share the weight at each step and every update copies the noise of a single sample; with a quarter of it, 114 do. A study of all four combinations of the last two decisions over 20 random seeds shows how they interact. The circling failure, which stops 4 of 20 runs, needs both the flawed cost and the erratic update, and with both corrections every run finishes, in 16.3 s on average and 16.5 s at worst, never closer than 0.79 m to an obstacle. A control step takes about 2.8 ms in vectorised NumPy.**

***Index Terms*—*model predictive control, path integral control, sampling-based control, autonomous vehicles, obstacle avoidance, cost function design*

I. Introduction

Model predictive control (MPC) repeatedly solves a finite-horizon optimal control problem from the current state, applies only the first command, and solves again at the next step [1]. Classical MPC relies on a model that is linear, or linearised around a reference, and on costs and constraints that keep the optimisation convex, so that a quadratic program can be solved within the control period. Obstacle avoidance breaks these assumptions: the set of collision-free positions is not convex, and the natural penalty for a collision is a step function with no useful gradient.

Model Predictive Path Integral (MPPI) control approaches the same receding-horizon problem from a different direction [2], [3]. Instead of optimising, it samples: every control step draws a few hundred random variations of its current plan, simulates the system forward with each one, scores the simulated trajectories with an arbitrary cost, and moves the plan toward the samples with low cost, each weighted by an exponential of its cost. The derivation comes from path integral control, in which the optimal control of a stochastic system can be written as an expectation over sampled trajectories [4], [5]. In practice the method is simple, parallelises well, and has driven a scale model car at the limits of handling on a dirt track [6].

This report describes a self-contained implementation of MPPI for a car-like robot that must follow a course and pass two obstacles placed on it. Its contributions are practical rather than theoretical:

- a compact, vectorised implementation of MPPI for a kinematic bicycle model, released with tests that check its behaviour rather than only running it;
- a control parameterisation that samples the steering rate, which bounds the steering motion between steps by construction;
- an analysis of a failure of a seemingly reasonable cost, in which the car circles beside an obstacle, together with its root cause and a one-line correction;

- an analysis of the weight of the control cost, which with its originally derived value collapses the effective number of samples to about two, and its correction;
- a factorial robustness study of the last two decisions over 20 random seeds, and measurements of runtime and of the effective number of samples.

II. Related Work

Path integral control was introduced by Kappen, who showed that for a class of stochastic optimal control problems with control-affine dynamics and a particular relation between noise and control cost, the Hamilton-Jacobi-Bellman equation becomes linear after an exponential transformation, and its solution can be estimated by sampling [4]. Theodorou, Buchli and Schaal generalised the idea into the PI² algorithm for learning parameterised policies [5]. Williams, Aldrich and Theodorou turned the sampling estimate into a receding-horizon controller, MPPI, and showed that it can be computed in real time on a graphics processor [2]. A later information-theoretic derivation removed the control-affine assumption by viewing the method as importance sampling of the optimal distribution over control sequences [3], [6]; the journal version also separates the weight of the control cost from the temperature [6], which Section IV-C shows to be essential here.

MPPI belongs to a broader family of sampling-based trajectory optimisers. The cross-entropy method refits a sampling distribution to the best-scoring fraction of samples rather than weighting all of them [7]; Pinneri et al. showed that drawing temporally correlated noise improves sample efficiency for such planners [8]. For vehicles, the alternatives are gradient-based MPC with a linearised kinematic or dynamic bicycle model [9], [10], and geometric or optimisation-based path planners followed by a tracking controller [11]. Gradient-based MPC handles constraints exactly but needs differentiable costs and convex approximations of obstacles; MPPI handles arbitrary costs but only satisfies constraints through the cost and the sampling, which is why the choices of cost, of control parameterisation and of weighting, studied here, decide its behaviour.

III. Problem Formulation

A. Vehicle model

The car is a kinematic bicycle model [9], [10] with state $x = (p_x, p_y, \theta, v, \delta)$: the position of the rear axle, the heading, the speed and the steering angle. The steering angle is a state rather than an input. The inputs are the acceleration a and the steering rate ω, both clipped to their limits. With the time step Δt and the wheelbase L, one step of the model is

$$p_{x,t+1} = p_{x,t} + v_t \cos\theta_t \, \Delta t, \qquad p_{y,t+1} = p_{y,t} + v_t \sin\theta_t \, \Delta t \tag{1}$$

$$\theta_{t+1} = \theta_t + \frac{v_t}{L} \tan\delta_{t+1} \, \Delta t, \qquad v_{t+1} = v_t + a_t \Delta t, \qquad \delta_{t+1} = sat(\delta_t + \omega_t \Delta t) \tag{2}$$

where sat clips the steering angle to $|\delta| \le \delta_{\max}$. The body is covered by a circle of radius $r_c = 1.7$ m centred half a wheelbase ahead of the rear axle; obstacles are discs with centre o_j and radius r_j, so the free distance between the car and obstacle j is $g_j = \|c - o_j\| - r_j - r_c$, where c is the centre of the covering circle. A negative g_j is a collision.

B. Task

The course is a natural cubic spline through six waypoints, parameterised by the distance between them and sampled every 0.5 m. Two obstacles of radius 1.0 m and 1.2 m sit on the course, 0.4 m and 0.5 m off its centre line, so that each has a preferred side. The car starts at rest at the start of the course and

must reach its end at a target speed of $v^* = 6$ m/s without touching an obstacle, within 40 s of simulated time.

C. Trajectory cost

A sampled control sequence produces states $x_1, \dots, x_T$ over the horizon. Its cost is the sum over the horizon of four terms: the squared distance d_t from the rear axle to the nearest course point, the squared error of the speed along the course, a squared penalty for entering a clearance margin m around each obstacle, and a fixed cost C for every step spent inside an obstacle:

$$S = \sum_{t=1}^{T} \left[w_c d_t^2 + w_v \left(v_t^{\parallel} - v^* \right)^2 + w_o \sum_j \max\left(0, m - g_{t,j}\right)^2 + C \cdot c_t \right] \tag{3}$$

Here c_t is one when any $g_{t,j}$ is negative and zero otherwise. The speed along the course is the component of the velocity in the direction ψ_t of the course at its nearest point:

$$v_t^{\parallel} = v_t \cos(\theta_t - \psi_t) \tag{4}$$

Section VI-B shows that this projection is not a detail: with the raw speed v_t in its place, the cost has a cheap and unwanted minimum.

IV. Method

A. Path integral update

MPPI keeps a nominal control sequence $\bar{u}_0, \dots, \bar{u}_{T-1}$. Every control step it draws K noise sequences ε^k with $\varepsilon_t^k \sim N(0, \Sigma)$, simulates the model from the current state with the controls $\bar{u}_t + \varepsilon_t^k$, and evaluates the cost S_k of each sample. In the information-theoretic derivation [3], the optimal distribution over control sequences V is proportional to the prior distribution reweighted by the exponentiated cost,

$$q^*(V) = \frac{1}{\eta} \exp\left(-\frac{1}{\lambda} S(V) \right) p(V) \tag{5}$$

with a normalising constant η and a temperature λ. MPPI estimates the mean of this distribution by importance sampling with the sampled sequences. Each sample receives the weight

$$w_k = \frac{\exp\left(-\frac{S_k - \rho}{\lambda} \right)}{\sum_{j=1}^{K} \exp\left(-\frac{S_j - \rho}{\lambda} \right)}, \qquad \rho = \min_k S_k \tag{6}$$

and the nominal sequence moves to the weighted mean of the samples:

$$\bar{u}_t \leftarrow \bar{u}_t + \sum_{k=1}^{K} w_k \varepsilon_t^k \tag{7}$$

Subtracting the smallest cost ρ does not change the weights, but it gives the best sample the unnormalised weight $\exp(0) = 1$, so the sum in the denominator can never underflow to zero, however large the costs are. Because the prior $p(V)$ is centred at zero control while the samples are drawn around the plan, the derivation also adds a control cost to each sample, with a weight γ:

$$S_k \leftarrow S_k + \gamma \sum_{t=0}^{T-1} \bar{u}_t^{\top} \Sigma^{-1} \varepsilon_t^k \tag{8}$$

In the original derivation $\gamma = \lambda$ [2], [3]; Section IV-C explains why this implementation uses $\gamma = \frac{\lambda}{4}$. After the update, the first command $\bar{u}_0$ is applied, the sequence is shifted by one step, and its last element is repeated to fill the horizon. Algorithm 1 summarises one control step.

Algorithm 1. One MPPI control step

1: Draw K noise sequences ε^k with $\varepsilon_t^k \sim N(0, \Sigma)$, for t = 0, ..., T-1.
2: Simulate the model from the current state with the controls $\bar{u}_t + \varepsilon_t^k$ (all samples at once).
3: Evaluate the cost S_k of every sampled trajectory, plus the control cost weighted by γ.
4: Compute the weights w_k from the costs, after subtracting the smallest cost.
5: Update the plan: $\bar{u}_t \leftarrow \bar{u}_t + \sum_k w_k \varepsilon_t^k$, then clip it to the input limits.
6: Apply $\bar{u}_0$ to the car, shift the plan by one step and repeat its last element.

B. Temperature and effective sample size

The temperature λ sets how many samples share the weight. As $\lambda \to 0$ the update copies the noise of the single best sample; as $\lambda \to \infty$ all samples receive equal weight and the update averages the noise to nothing. A convenient measure is the effective sample size

$$ESS = \frac{1}{\sum_{k=1}^{K} w_k^2} \tag{9}$$

which equals K for uniform weights and one when a single sample carries all the weight. An effective sample size close to one is a warning sign: the update then follows the random draw of one sample rather than an average, and consecutive control steps behave like independent random choices.

C. The weight of the control cost

The control cost of (8) is linear in the noise. For the samples of one control step, its contribution to the exponent of the weights is therefore a Gaussian random variable with zero mean and standard deviation

$$\sigma_{\text{ctrl}} = \frac{\gamma}{\lambda} \sqrt{\sum_t \left\| \Sigma^{-\frac{1}{2}} \bar{u}_t \right\|^2} \tag{10}$$

the distance of the plan from zero, measured in units of the noise and multiplied by $\frac{\gamma}{\lambda}$. With $\gamma = \lambda$ the ratio is one, and the temperature cancels: the spread that this term adds to the exponents is fixed by the plan alone. In the demonstration task with $\gamma = \lambda$, the plan lies a median of 6.1 noise widths from zero, measured over the whole horizon, so this term alone makes the weights of the samples differ by factors of the order of $e^6 \approx 400$, and a few samples take nearly all of the weight: the median effective sample size is 2.6 of 300. The effect feeds itself. Each update copies the noise of one sample into the plan, which moves the plan further from zero and widens the spread at the next step. Every random seed then drives its own path, and floating-point differences between platforms act like a change of seed.

Williams et al. separate the two weights, with $\gamma = \lambda(1 - \alpha)$ for a parameter α between zero and one [6]. This implementation uses $\gamma = 5$, a quarter of $\lambda = 20$. The plan then stays closer to zero, at a median of 2.7 noise widths, the spread σ_{ctrl} falls to about 0.7, and the median effective sample size rises to 114 of 300; repeated updates from the same state with different random draws produce nearly the same command (Section V).

D. Sampling the steering rate

A first version, which also used $\gamma = \lambda$, sampled the steering angle directly, with independent noise at every step. Since each control step takes a fresh weighted average of fresh noise, the applied steering angle jumped by a large part of the noise standard deviation from one step to the next: about 6 degrees

per step, visible as a wobbling path. Sampling time-correlated noise did not help, because the first element of each new plan is still a fresh average. Moving the steering angle into the state and sampling its rate solves the problem at its source. The steering angle now changes by at most $\omega_{\max}\Delta t = 4$ degrees per step, whatever the sampled noise, and a random walk on the rate explores sustained turns, which independent noise on the angle rarely does.

V. Implementation

The controller is a single Python module built on NumPy [12] and SciPy [13]; it grew out of the PythonRobotics collection [14] and follows its conventions, so running the module animates the car as it drives. All K samples are simulated at once as arrays of shape $(K, T, 5)$, so one control step consists of T vectorised model evaluations and a handful of array operations. The squared distances from all $K \cdot T$ sampled positions to a window of 60 course points ahead of the car are computed with a single matrix product, using $\|p - q\|^2 = \|p\|^2 - 2p \cdot q + \|q\|^2$; a first version that formed the $(K \cdot T, 60, 2)$ array of differences explicitly spent 85% of the step time there. Table I lists the parameters.

TABLE I
Parameters of the controller and the vehicle

Parameter	Symbol	Value
Time step	Δt	0.1 s
Horizon	T	25 steps (2.5 s)
Samples per step	K	300
Temperature	λ	20
Weight of the control cost	γ	5
Noise standard deviation	$\Sigma^{\frac{1}{2}}$	1.0 m/s², 20 °/s
Wheelbase	L	2.0 m
Steering limits	$\delta_{\max}$, $\omega_{\max}$	35°, 40 °/s
Acceleration limit	$a_{\max}$	3.0 m/s²
Target speed	v^*	6 m/s
Cost weights	w_c, w_v, w_o, C	1, 1, 200, 10^4
Clearance margin	m	1.0 m

The software is tested by eleven automated tests. Beyond checking that the demonstration finishes, they compare the fast distance computation with a brute-force search, check the weight computation on hand-made cases (including costs too large for the exponential to represent), assert that driving across the course at full speed costs exactly as much as standing still, and run the two random seeds that exposed the failure described in Section VI-B as regression tests. One test repeats the MPPI update from three states of the demonstration with ten different random draws: the first commands they produce spread by 0.11 noise standard deviations, against about 0.6 if each update copied the noise of a single sample. Every bound in the tests holds with margin over seeds 0 to 99, because floating-point differences between platforms change a run in the same way as a different seed does.

VI. Experiments

A. Demonstration run

Fig. 1 shows the run of the demonstration (seed 3). The car leaves the course to pass each obstacle with the clearance margin intact and returns to it afterwards; it finishes in 16.3 s, comes no closer than 0.80 m to an obstacle, and away from the obstacles stays within 0.39 m of the course. Fig. 3 shows the speed and steering angle: the car accelerates smoothly to the target speed, slows slightly between the obstacles, and steers by at most 20.2°, at a rate of at most 30 °/s, below the 40 °/s limit.

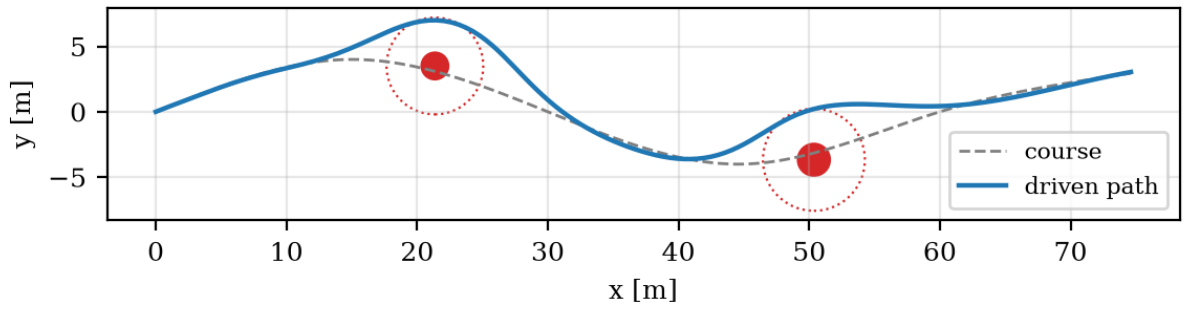


Fig. 1. Course (dashed), obstacles with their clearance margins (dotted) and the path driven by the car in the demonstration run.

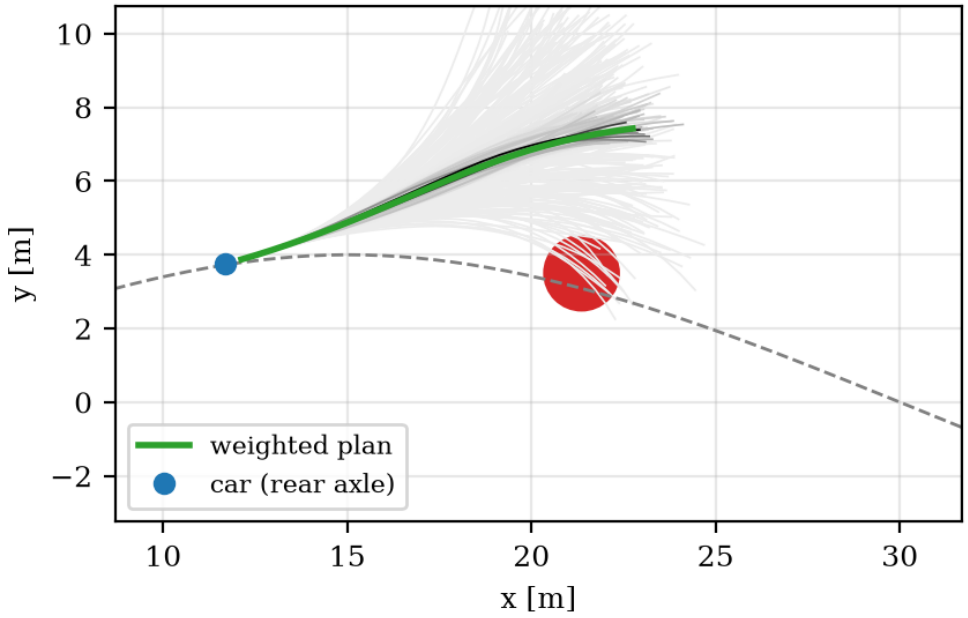


Fig. 2. One control step 9 m before the first obstacle. Grey lines: the 300 sampled trajectories, darker for larger weight. Green: the plan after the update, which passes the obstacle on its far side. Samples that run into the obstacle receive weights of essentially zero.

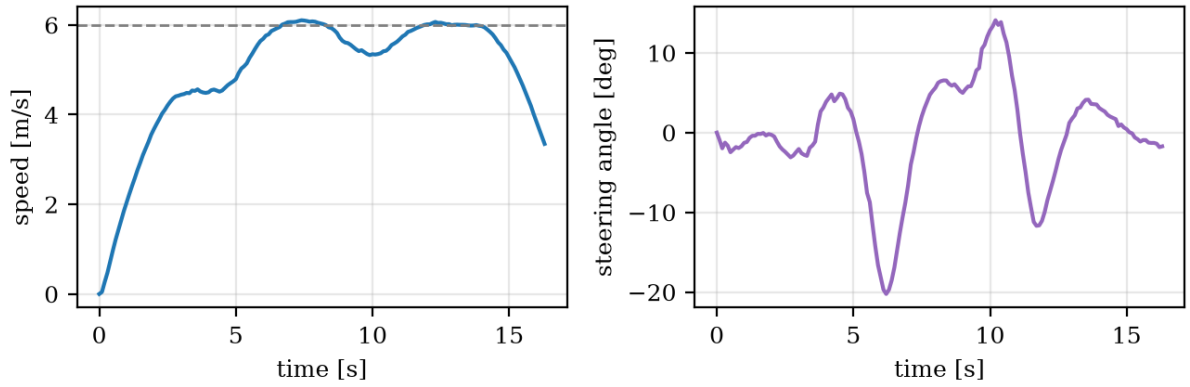


Fig. 3. Speed (left) and steering angle (right) over time in the demonstration run. The dashed line is the target speed.

B. The circling failure and its root cause

The first version of the cost used the raw speed v_t instead of the speed along the course, together with the control cost weight $\gamma = \lambda$. With it, the car finished the demonstration, but a study over eleven seeds showed that two runs never reached the end of the course. The obvious explanation was that slowing down in front of an obstacle cost less than the detour around it, and a natural remedy was to make detours cheaper with a Huber-type course cost. That made matters worse: seven of eleven runs failed. Instrumenting a failed run showed why. The car was not stopped at all; it was driving at full speed with the steering at its 35° limit, circling beside the first obstacle. A circle of radius $\frac{L}{\tan \delta_{\max}} \approx 2.9$ m next to the course crosses the course twice per revolution, so the course distance stays small, and it is driven at the target speed, so the speed term is zero. Nothing in the cost asked the car to make progress.

Counting only the speed along the course removes this minimum: a circle has an along-course speed that averages to zero, so it pays roughly $w_v v^{*2} = 36$ per step, far more than the detour.

C. Interaction of the cost and the control-cost weight

The two corrections address different parts of the method: the speed along the course removes an unwanted minimum of the cost, and the reduced weight γ makes the update an average over many samples rather than a copy of one. To separate their effects, all four combinations were run with 20 random seeds. Table II and Fig. 4 show the results.

TABLE II
ALL FOUR COMBINATIONS OF THE SPEED TERM AND THE CONTROL-COST WEIGHT, OVER 20 RANDOM SEEDS

Speed in the cost	Control-cost weight	Runs finished	Mean time	Worst time	Worst clearance	Worst course distance	Median ESS
Raw speed	$\gamma = \lambda$	16 of 20	24.2 s	36.4 s	0.55 m	2.63 m	2.5
Along the course	$\gamma = \lambda$	20 of 20	19.7 s	20.9 s	0.62 m	2.57 m	2.6
Raw speed	$\gamma = 5$	20 of 20	17.2 s	17.4 s	0.80 m	0.55 m	140
Along the course (used)	$\gamma = 5$	20 of 20	16.3 s	16.5 s	0.79 m	0.51 m	114

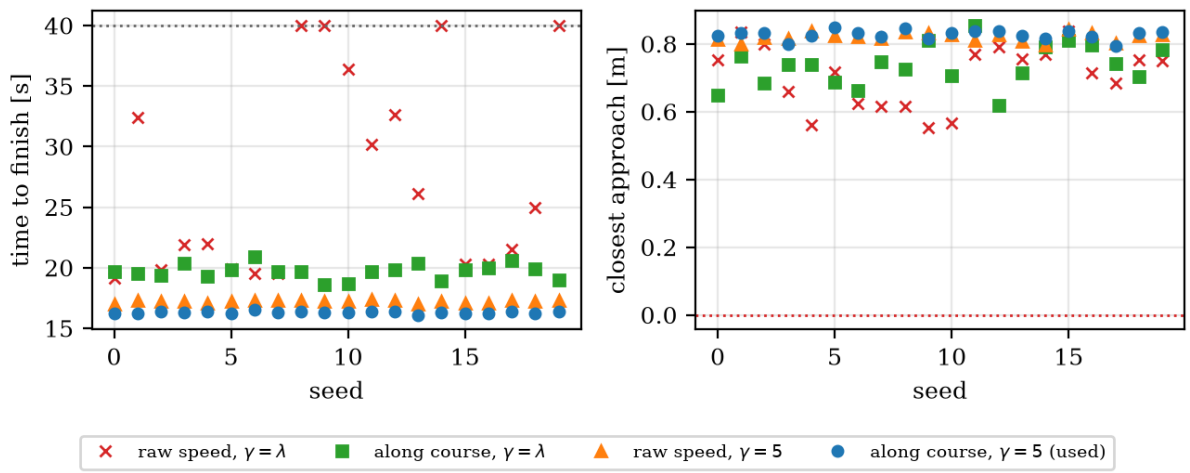


Fig. 4. Time to finish (left; 40 s means the run never finished) and closest approach to an obstacle (right) for 20 random seeds, for the four combinations of the speed term and the control-cost weight.

The table shows that the circling failure needs both defects. With $\gamma = \lambda$ the effective sample size is about 2.5 whatever the cost, the updates follow the random draws of single samples, and the car wanders: with the raw speed it falls into the circling minimum in 4 of 20 runs, and even with the corrected cost its runs vary by more than 2 s and stray up to 2.6 m from the course. With $\gamma = 5$ about a hundred samples share the weight, the runs become nearly identical, and the car no longer stumbles into the unwanted minimum even when the cost contains it. The corrected cost still pays off: with it the car finishes 0.9 s earlier on average, driving at 4.9 m/s on average against 4.7 m/s. Both corrections are therefore worth having, for different reasons: the first makes the optimum of the cost the intended behaviour, and the second makes the optimiser find it reliably.

D. Runtime and sample efficiency

A control step takes 2.8 ms on average on a laptop processor (163 steps in 0.45 s), well within the 100 ms control period, so the method would run in real time even with several times more samples. Over the demonstration run, the median effective sample size is 113 of 300; at the control step shown in Fig. 2, just before the first obstacle, where some samples collide and receive essentially no weight, it is 39. Table III summarises the demonstration run.

TABLE III
DEMONSTRATION RUN (SEED 3)

Measure	Value
Time to finish	16.3 s (163 control steps)
Closest approach to an obstacle	0.80 m
Largest distance from the course away from obstacles	0.39 m
Largest steering angle and steering rate	20.2°, 30 °/s
Runtime per control step	2.8 ms
Effective sample size: median over the run, at the step of Fig. 2	113 and 39 of 300

VII. DISCUSSION

The three decisions examined in this report share a theme: MPPI optimises exactly what the cost, the parameterisation and the weighting say, and nothing else. Sampling the steering angle made the applied steering jumpy because nothing in the method links consecutive control steps except the shifted plan; sampling the rate builds that link into the model, where it holds by construction instead of by penalty.

The circling failure arose because the cost rewarded speed without direction. The collapse of the effective sample size arose because a term derived for a prior centred at zero control was given a weight that made it dominate the weights whenever the plan was far from zero. None of these problems is visible in a single demonstration run, which succeeded with each of the flawed variants; they appear only when the method is run many times with different random numbers. Evaluating a randomised controller over many seeds, monitoring its effective sample size, and testing the cost function itself on hand-made trajectories should be routine.

The temperature and the control-cost weight play different roles, and the study shows why they should be set separately. The temperature trades how strongly the update follows the best samples against how much it averages; the control-cost weight sets how strongly the plan is pulled back towards zero control. With $\gamma = \lambda$, raising the temperature cannot restore the effective sample size, because the spread of the control-cost term in the exponents does not depend on it; only a smaller ratio $\frac{\gamma}{\lambda}$ can. An adaptive temperature, chosen each step to reach a target effective sample size, would remove the remaining tuning and is a natural extension.

The study has limitations. The vehicle model is kinematic, so it ignores tyre slip, which matters at higher speeds [10]; the obstacles are static and known exactly; and clearance is enforced only through the cost, so MPPI offers no guarantee of safety. In a deployed system the sampled controller would typically be combined with a safety layer that does give guarantees, such as a control barrier function filter.

VIII. Conclusion

This report presented an implementation of MPPI control for a car-like robot that follows a course and avoids obstacles, together with tests and a reproducible evaluation. Three design decisions proved decisive: sampling the steering rate rather than the angle, which makes the steering smooth by construction; measuring speed along the course rather than raw speed, without which the cost rewards circling beside an obstacle; and weighting the control cost by a quarter of the temperature rather than the full temperature, without which a single sample dominates every update. With all three, the controller finishes the course in every one of 20 random runs, within 16.5 s and never closer than 0.79 m to the obstacles, at 2.8 ms per control step. The code, the tests and the script that regenerates every figure and number in this report are available at github.com/robofan45/mppi-control.

Acknowledgment

The software and this report were developed with the assistance of an AI coding assistant (Claude, by Anthropic). All results were produced by the code in the repository and can be regenerated with scripts/make_figures.py.

References

[1] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.

[2] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *J. Guid. Control Dyn.*, vol. 40, no. 2, pp. 344–357, 2017.

[3] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 1714–1721.

[4] H. J. Kappen, "Linear theory for control of nonlinear stochastic systems," *Phys. Rev. Lett.*, vol. 95, no. 20, Art. no. 200201, 2005.

[5] E. Theodorou, J. Buchli, and S. Schaal, "A generalized path integral control approach to reinforcement learning," *J. Mach. Learn. Res.*, vol. 11, pp. 3137–3181, 2010.

[6] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1603–1622, 2018.

[7] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method: A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation and Machine Learning*. New York, NY, USA: Springer, 2004.

[8] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, and G. Martius, "Sample-efficient cross-entropy method for real-time planning," in *Proc. Conf. Robot Learn. (CoRL)*, 2020.

[9] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[10] J. Kong, M. Pfeiffer, G. Schildbach, and F. Borrelli, "Kinematic and dynamic vehicle models for autonomous driving control design," in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2015, pp. 1094–1099.

[11] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Trans. Intell. Veh.*, vol. 1, no. 1, pp. 33–55, 2016.

[12] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020.

[13] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, pp. 261–272, 2020.

[14] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Paques, "PythonRobotics: A Python code collection of robotics algorithms," arXiv:1808.10703, 2018.